CYBR 343 | CLASS PROJECT

AEGIS-OS

A secure operating system design for a nuclear facility



Daniel Guy

Why a nuclear plant needs its own secure OS



2010

Stuxnet

Malware crossed an air gap on a USB drive and physically wrecked uranium centrifuges in Iran.

AEGIS-OS has three goals



Only trusted code runs

Every program must carry a valid digital signature.



One broken part can't sink the rest

The system is split into sealed-off partitions.



Safety always comes first

The emergency shutdown stays hard-wired, outside any software.

Required by federal rules: 10 CFR 73.54 and NRC Regulatory Guide 5.71

How Stuxnet broke into an offline nuclear site

Natanz uranium enrichment plant, Iran

1

Got in on a USB drive

The plant had no internet connection, so the worm rode in on a USB stick.

2

Looked trustworthy

Its drivers were signed with stolen certificates, and it used four unknown Windows flaws.

3

Hijacked controllers

It found the Siemens controllers that ran the centrifuges and changed their code.

4

Lied to the operators

It sped the centrifuges up and down while replaying normal readings on screen.

June 2010

found by a small antivirus firm in Belarus

WHAT AEGIS-OS TAKES FROM IT

- USB storage is turned off
- Only the facility's own keys are trusted, and stolen ones can be blocked
- Safety shutdown stays hard-wired, so fake readings can't stop it

The result: code that broke real machines



THE SITE

Natanz enrichment plant, Iran



WHAT IT HIJACKED

Siemens S7-300 controller



WHAT IT BROKE

Gas centrifuges like these

~1,000

centrifuges destroyed and replaced, about 1 in 9 at Natanz

100,000+

computers infected worldwide, most of them in Iran

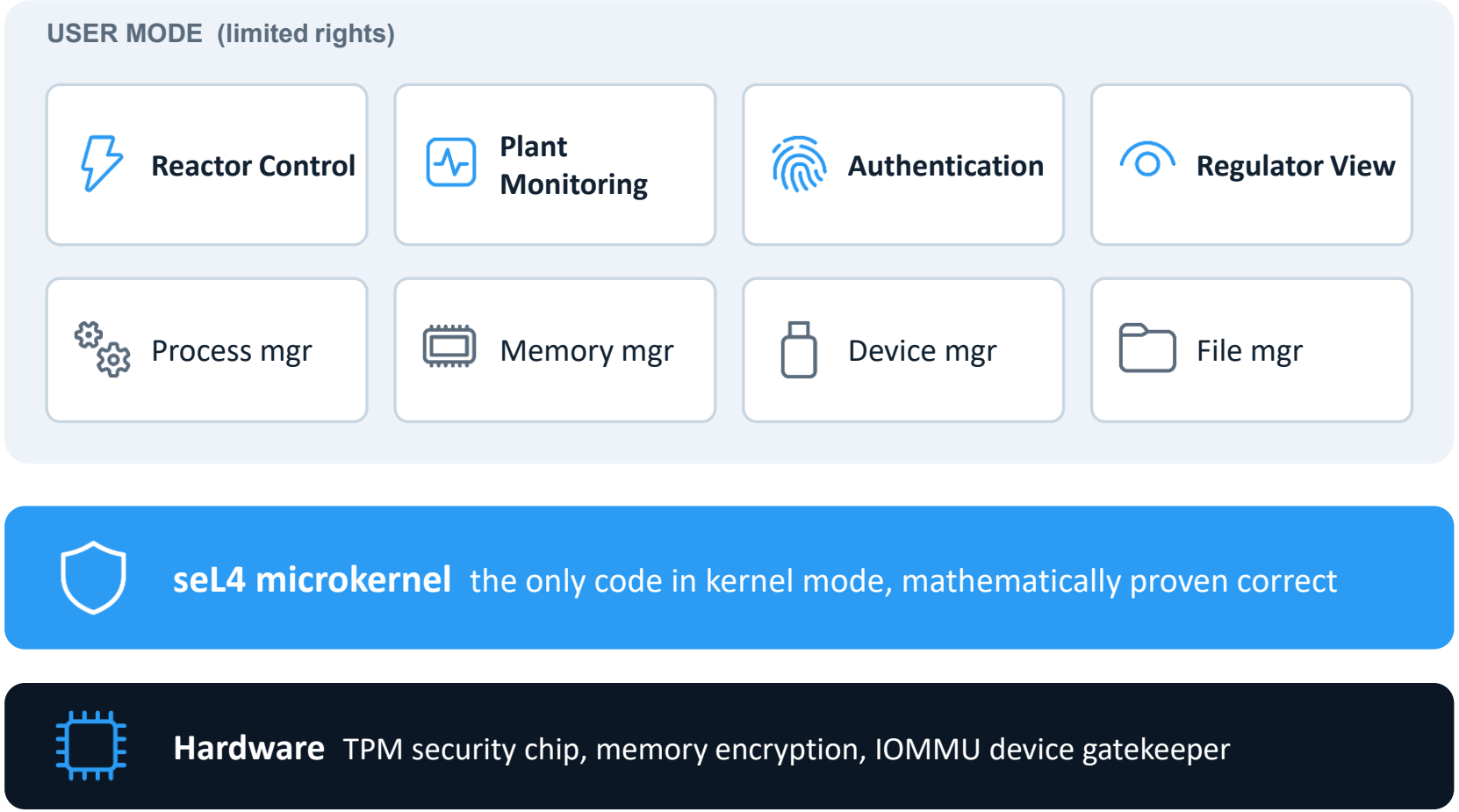
2011-12

copycat tools Duqu and Flame followed, built from related code

Photos via Wikimedia Commons: Parsa 2au (CC BY-SA 4.0); Ulli1105 (CC BY-SA 2.5); U.S. Department of Energy (public domain, U.S. plant shown).

One tiny kernel, four sealed-off partitions

Only the seL4 microkernel has full control. Everything else runs with limited rights.



Proven

seL4 has a math proof that its code does what its spec says.

Isolated

Partitions share no memory. They talk only through kernel-approved channels.

Small

Less code to trust means fewer places for bugs to hide.

Every part of the OS is locked down

Least privilege, fail-safe defaults, and layers of defense



Processes

Only signed programs start. Hard limits per partition stop runaway programs.



Memory

Encrypted per partition and wiped after use. ASLR and DEP block exploits.



Devices

Each device reaches only its own memory. Unknown USB devices are blocked.



Files

AES-256 encrypted disk. Audit logs can be added to, never edited.



People

Smart card, PIN, and fingerprint. Two people approve any safety action.

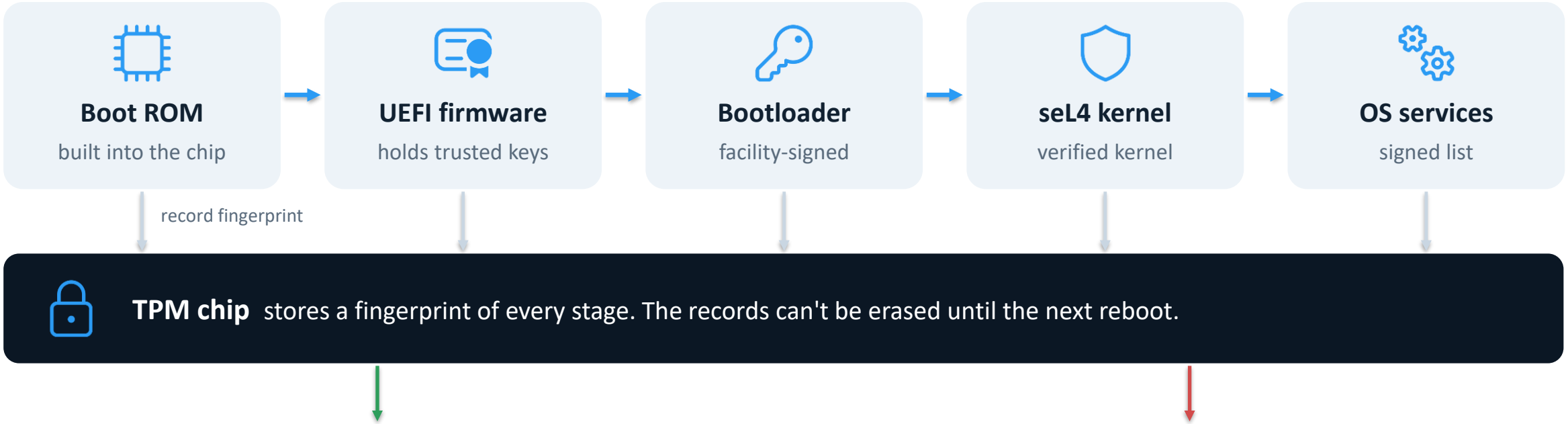


Network

A one-way data diode lets data out. Nothing can come back in.

Secure boot: every stage checks the next one

The foundation of the design. If startup can be changed, every other protection can be turned off.



Attacks the design shuts down

THREAT		HOW AEGIS-OS STOPS IT
Bootkit	malware that loads before the OS	Secure Boot refuses unsigned startup code
Rootkit	malware hiding in the kernel	Tiny proven kernel and signed drivers only
Buffer overflow	writing past a memory boundary	ASLR, DEP, and guard pages
DMA attack	a device reading memory directly	IOMMU limits each device to its own area
Rogue device	an unknown device plugged in	Unknown devices are blocked and logged
Fork bomb	a program copying itself endlessly	Hard process limits per partition

Demo: Secure Boot on vs. off

Ubuntu VM in VirtualBox with a virtual TPM chip, using the same mechanisms AEGIS-OS relies on

